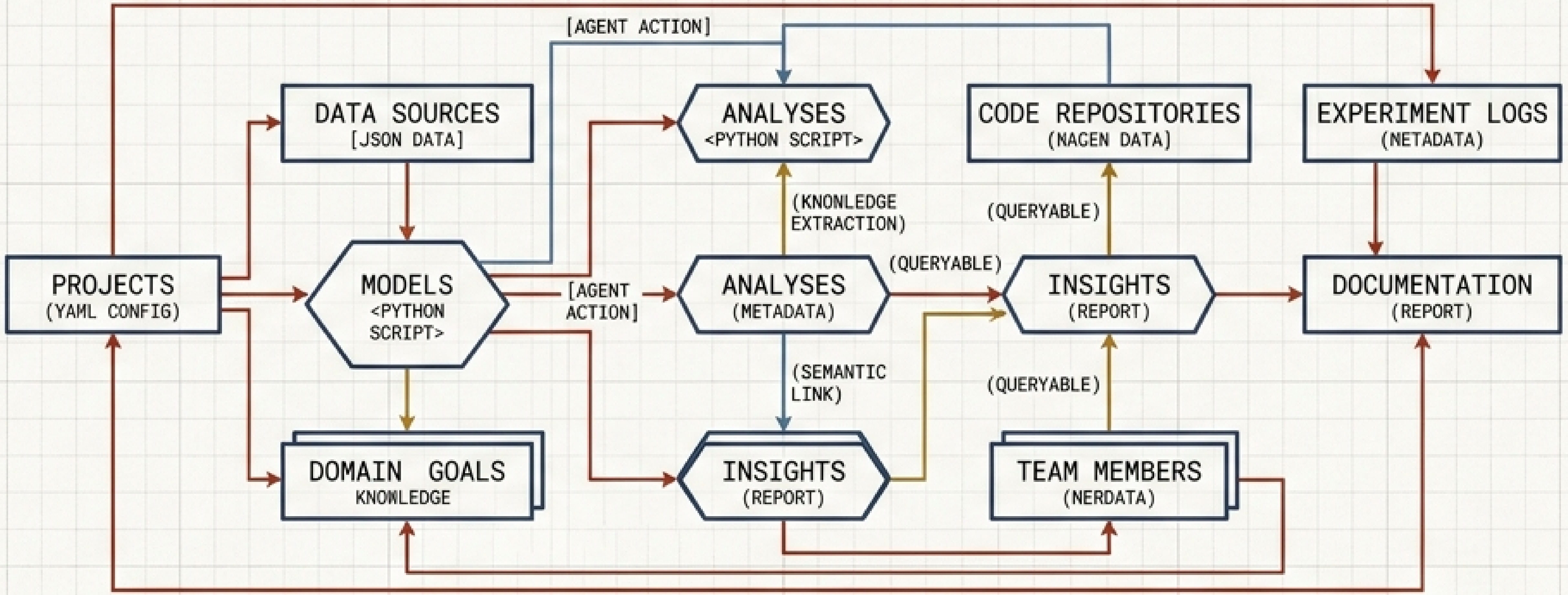
KNOWLEDGE MANAGEMENT FOR DATA SCIENCE (KMDS)

Transforming Analytical Projects into Queryable Institutional Knowledge

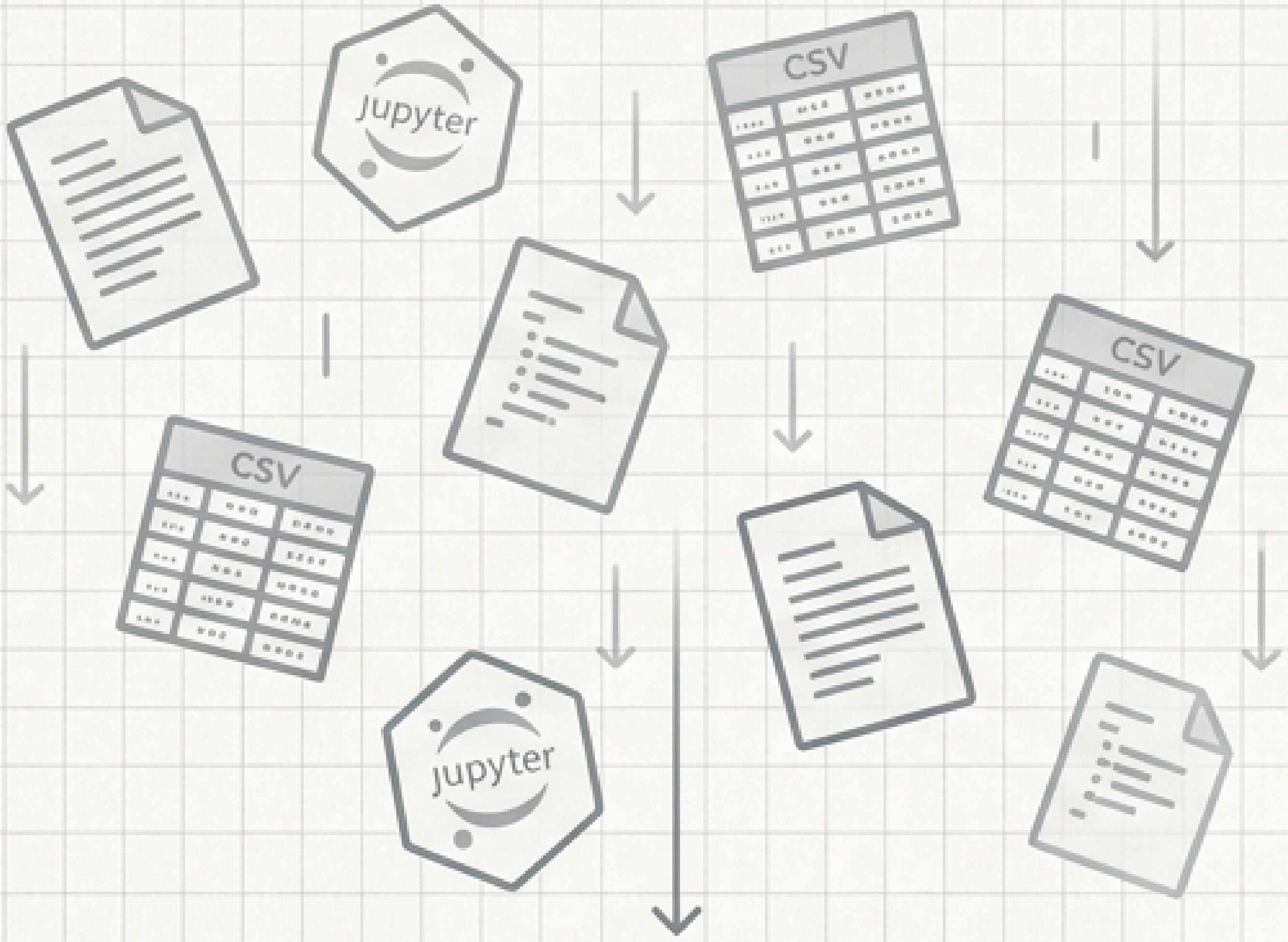


AGENT-DRIVEN, LOCAL-FIRST KNOWLEDGE GRAPHS FOR ENTERPRISE ML WORKFLOWS.

NotebookLM

The Problem: The Fragmented ML Context

Before: The Memory Hole



YAML

Experimental work generates a fragmented stream of insights. As systems evolve, understanding why a decision was made becomes harder than what decision was made.

JSON

After: Structured Institutional Knowledge

YAML

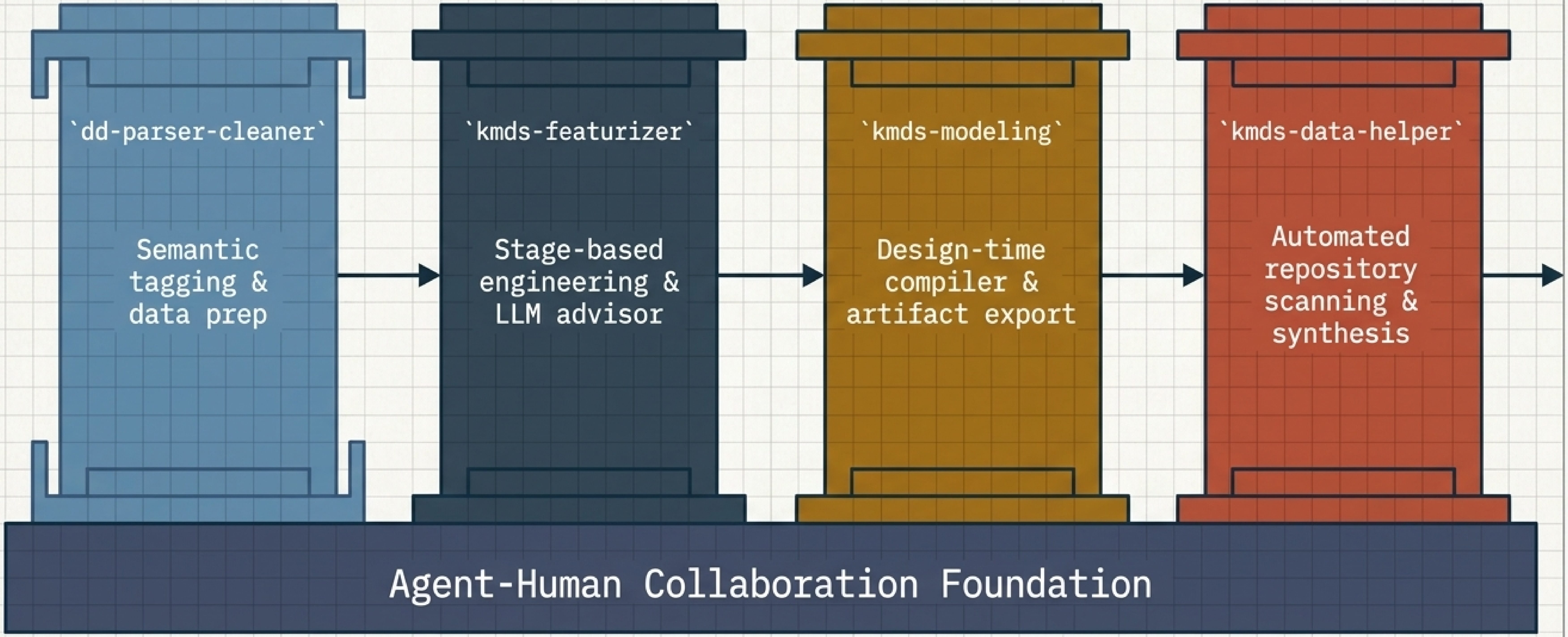
Institutional memory is preserved. Disparate documentation and isolated scripts are bound together into an accessible, queryable graph.

JSON

NotebookLM

The Solution: The KMDS Ecosystem

KMDS logs, maps, searches, and visually audits data engineering artifacts—generating RDF/XML knowledge graphs without data leaving the local machine.



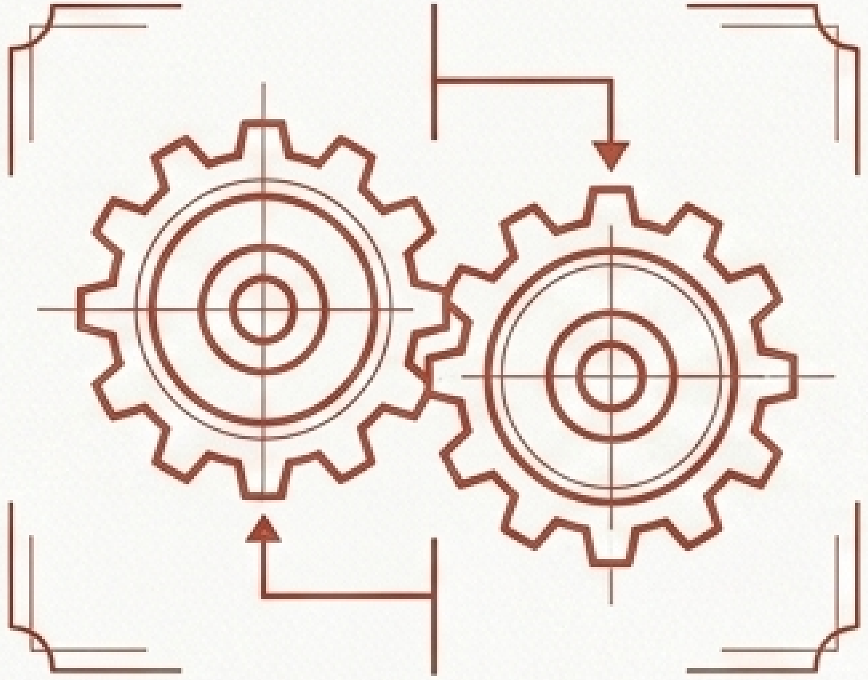
NotebookLM

Philosophy: Agent-Human Collaboration

The Human

Accountability & Expertise

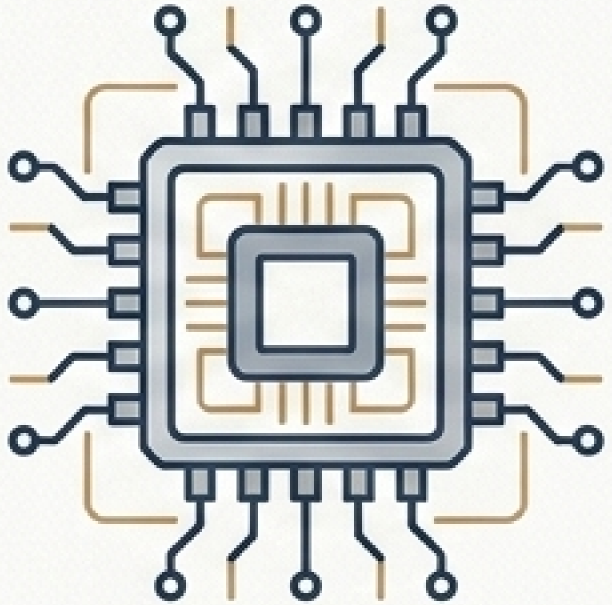
Retains absolute ownership of performance, accountability, and final decision-making. Evaluates edge cases and provides domain expertise.



The KMDS Protocol

The Handshake

Provides the structured interfaces, data contracts, and architectural guardrails that make collaboration repeatable and safe.



The LLM Agent

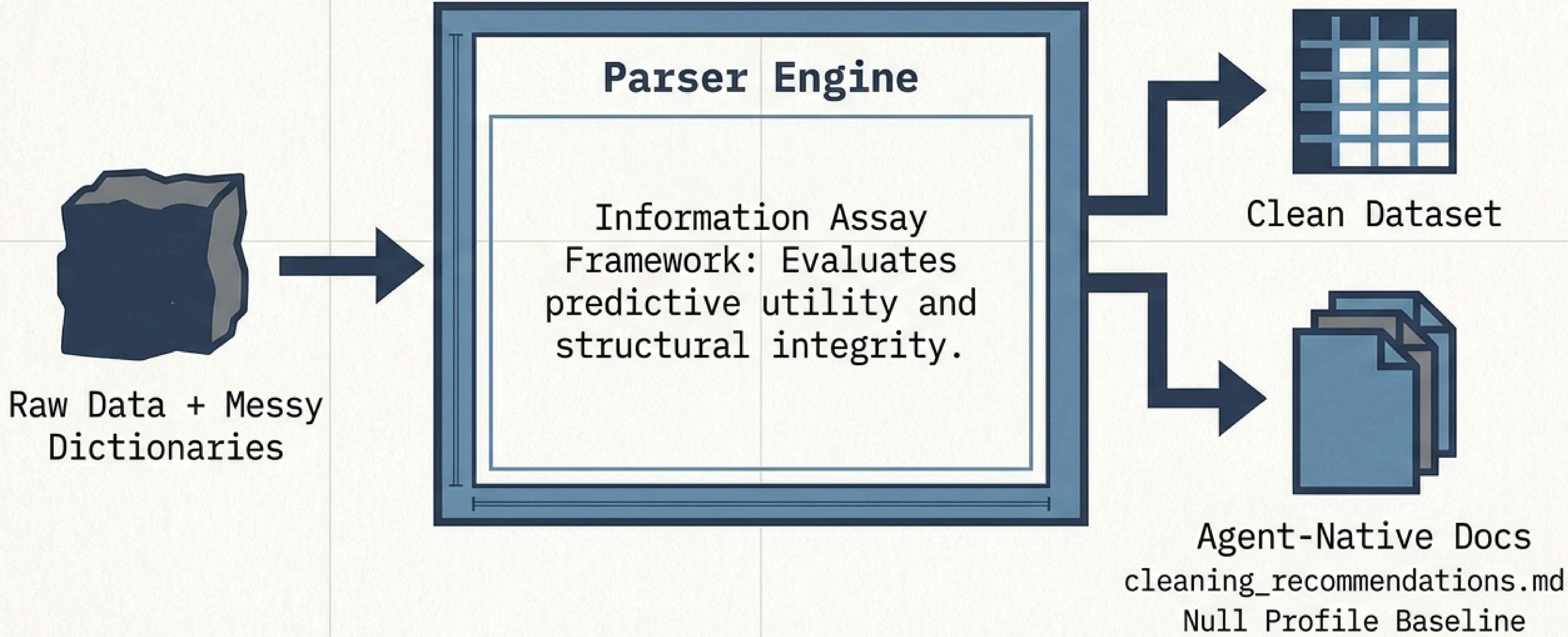
Efficiency & Enforcement

Retrieves context, generates boilerplate code, applies complex sequences, and ensures strict rule enforcement.

NotebookLM

Stage 1: The Semantic Foundation

dd-parser-cleaner



NotebookLM

The Data Prep Assembly Line

Distilling the 12-Step Operational Recipe into 4 Macro-Phases

1. Setup & Bootstrap

Initialize the workspace, locate files, and generate a provisional config.

`init-workspace`

2. Classify (The Handshake)

Synchronize metadata and tag semantic entities.

`classify-entities`

3. Clean (The Pipeline)

Run the diagnostic pipeline to apply types, filters, and transformations.

`clean-dataset --action full`

4. Review & Modify

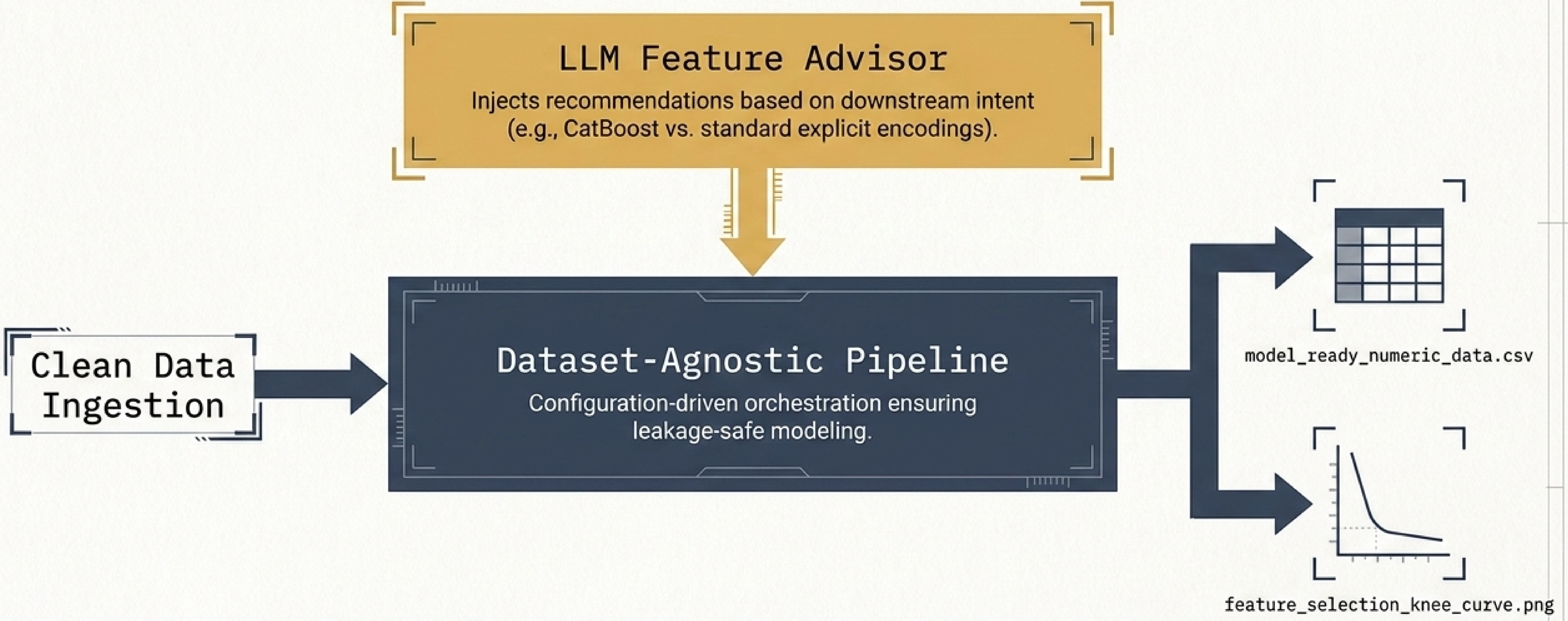
Review AI-driven recommendations and implement domain logic.

`Jupyter Notebooks`

NotebookLM

Stage 2: The Feature Advisor

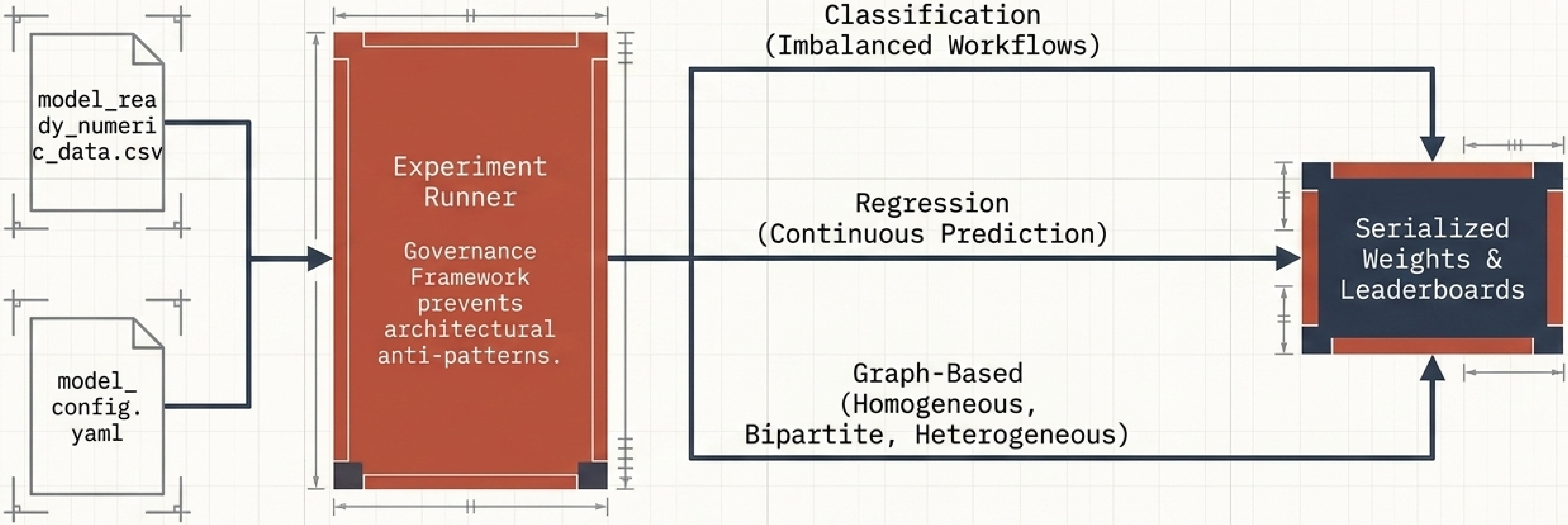
kmds-featurizer



NotebookLM

Stage 3: The Design-Time Compiler

kmds-modeling



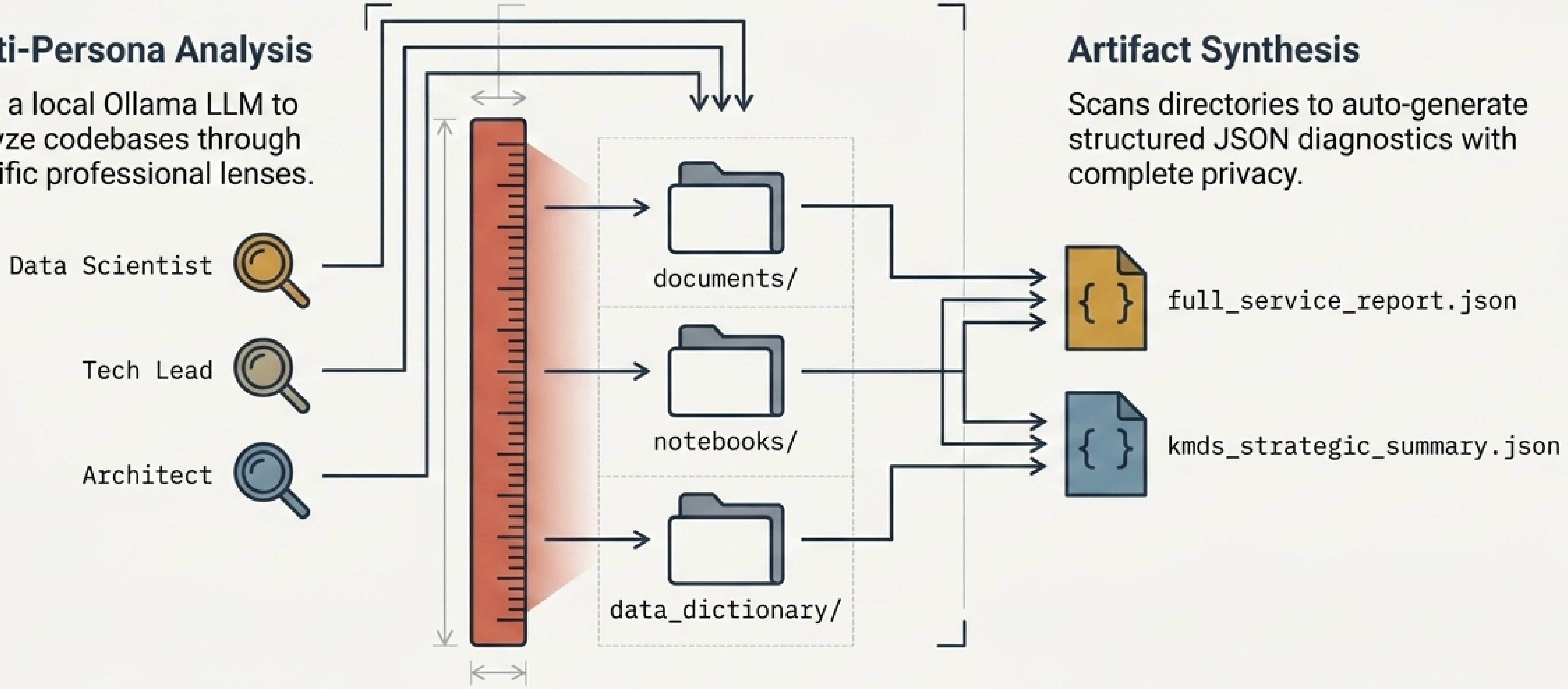
NotebookLM

Stage 4: Automated Repository Auditing

`kmds-data-helper`

Multi-Persona Analysis

Uses a local Ollama LLM to analyze codebases through specific professional lenses.



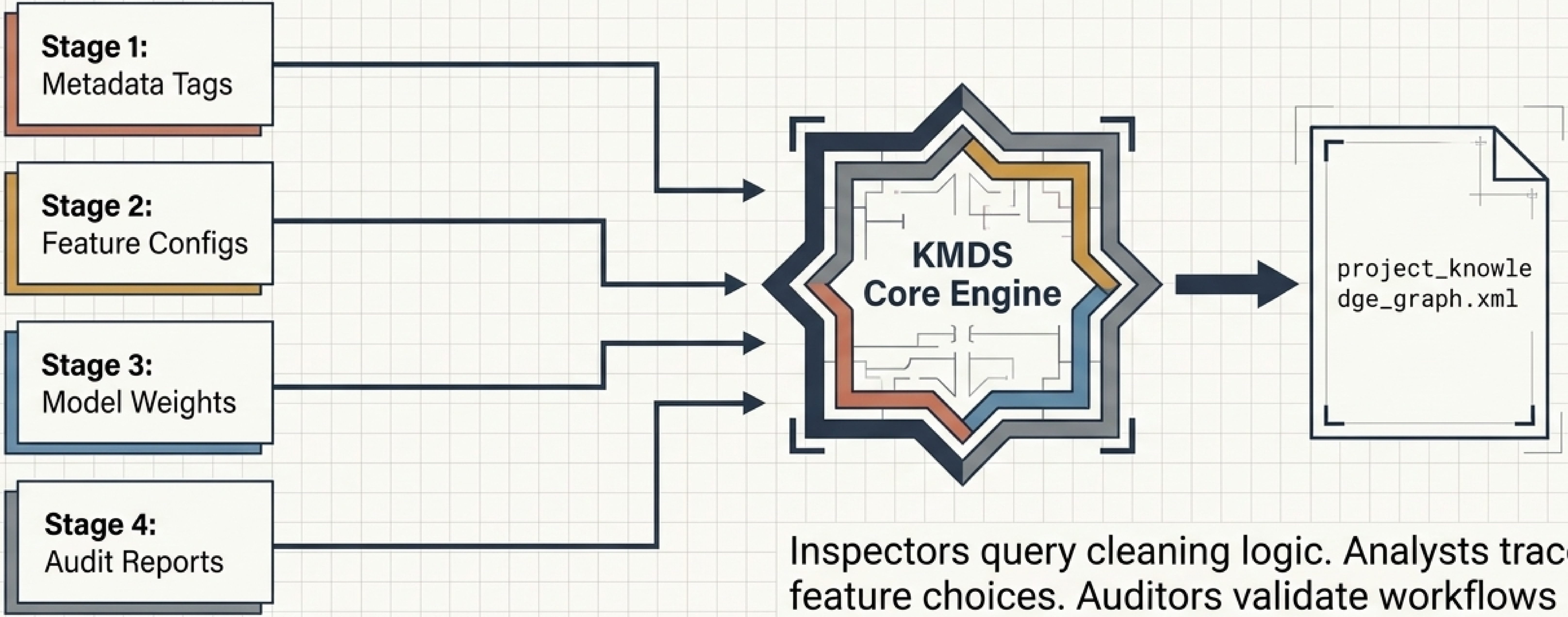
Artifact Synthesis

Scans directories to auto-generate structured JSON diagnostics with complete privacy.

NotebookLM

The Synthesis Engine: Crystallizing the Knowledge Graph

Converting the entire project history into an auditable, queryable structure.



Inspectors query cleaning logic. Analysts trace feature choices. Auditors validate workflows entirely locally.

NotebookLM

The Interactive UI Workbench

kmds-ui

Prefix-Agnostic Processing

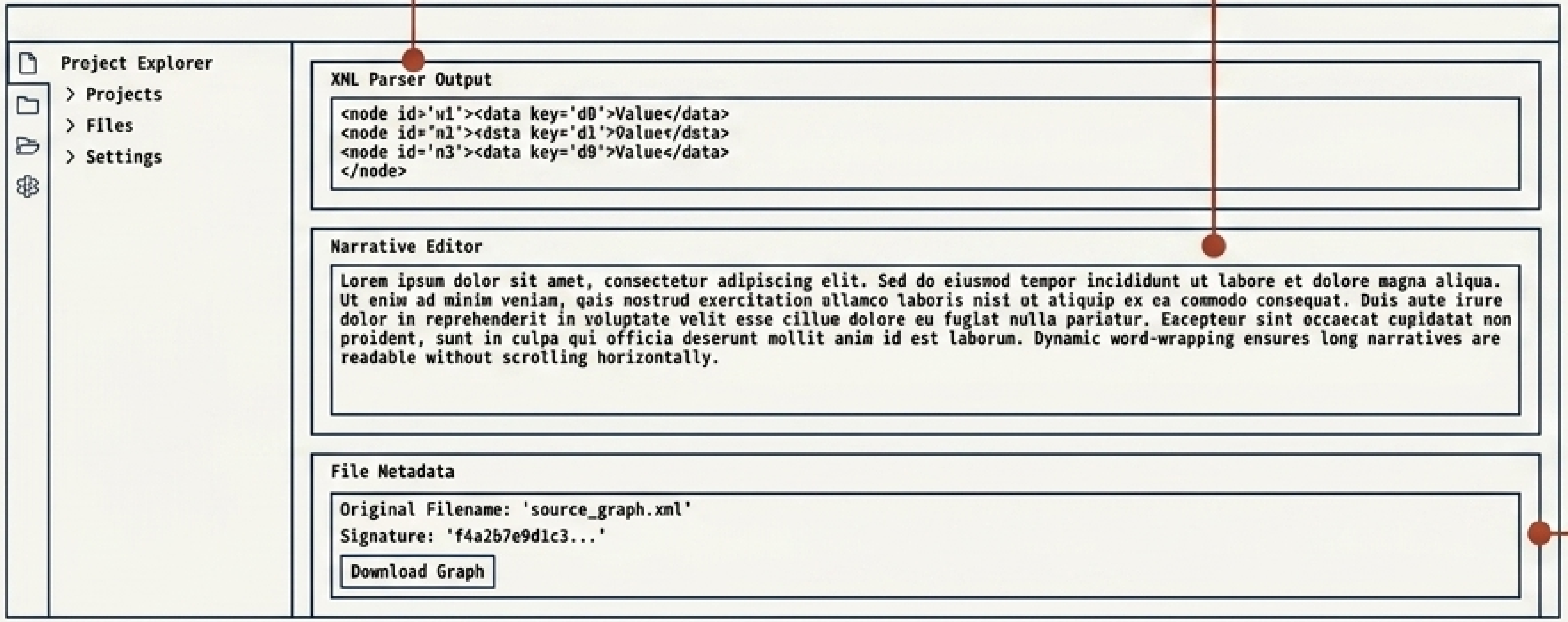
Dynamically parses XML fragments at runtime to handle files lacking explicit namespace declarations without crashing.

Proportional Narrative Isolation

Uses a 75% proportional grid with dynamic word-wrapping to cleanly display long text fields without truncation.

Preserved File Context

Tracks the original file name during ingestion, ensuring edited graphs download with matching signatures.



NotebookLM

Separation of Responsibilities Matrix

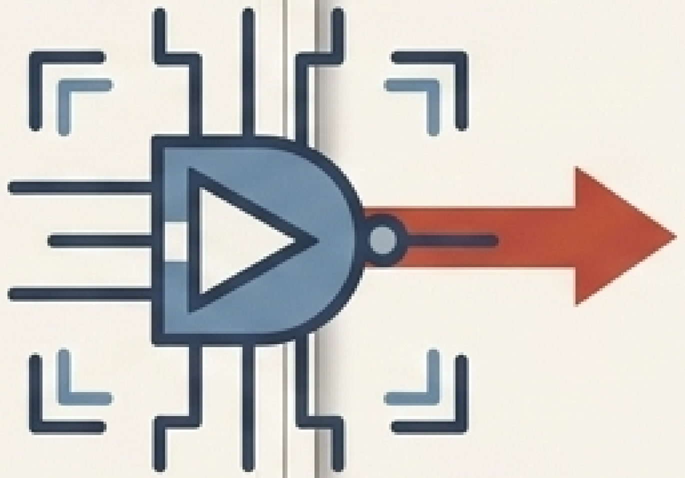
Phase	The Human	KMDS Packages	The Agent
Entity Selection	Chooses geographic & temporal entities	Offers components for logical types	Routes data, ensures entity types
Feature Engineering	Decides encoding strategy	Implements reusable transforms	Applies correct operational sequence
Modeling Workflow	Chooses algorithms & thresholds	Evaluation & artifact serialization	Orchestrates execution & export
Audit & Doc	Reviews model readiness	Metadata generation interfaces	Converts repo into auditable graph

NotebookLM

The Finished Product: The Design Blueprint

The workflow culminates in an **expert prompt** that guarantees AI assistants generate **production-grade code** adhering to governance rules.

```
EXPERT_PROMPT:
{
  "goal": "Generate production-grade Python code for a gradient
  boosting model pipeline.",
  "constraints": [
    "Adhere to PEP 8 guidelines.",
    "Include comprehensive docstrings and type hints.",
    "Ensure auditability via serialized artifact generation.",
    "Implement error handling and logging.",
    "No generic or placeholder variables."
  ],
  "input_spec": {
    "features": ["cluster_distance_metrics"],
    "target": "borrower_risk_score",
    "algorithm": "XGBoost"
  },
  "output_spec": {
    "format": "Serialized Python script (.py)",
    "artifact_path": "./artifacts/model_v1.pkl"
  }
}
```



Production Python Code

```
# Production Python Code
import xgboost as xgb
import pandas as pd
import joblib

import logging

logging.basicConfig(level=logging.INFO)

def train_model(data: pd.DataFrame, features: list[str], target: str,
output_path: str) -> None:
    """Trains a gradient boosting model and serializes the artifact."""
    try:
        X = data[features]
        y = data(target)
        model = xgb.XGBClassifier(use_label_encoder=False,
        eval_metric='logloss')
        model.fit(X, y)

        joblib.dump(model, output_path)
        logging.info(f"Model serialized to (output_path)")
    except Exception as e:
        logging.error(f"Error during training: (e)")

# Example Usage (Mock Data)
# data = pd.read_csv('sba_data.csv')
# train_model(data, ['vluster_distance_metrics'],
'borrower_risk_score', './artifacts/model_v1.pki')
```

Concrete Proof: The SBA Implementation

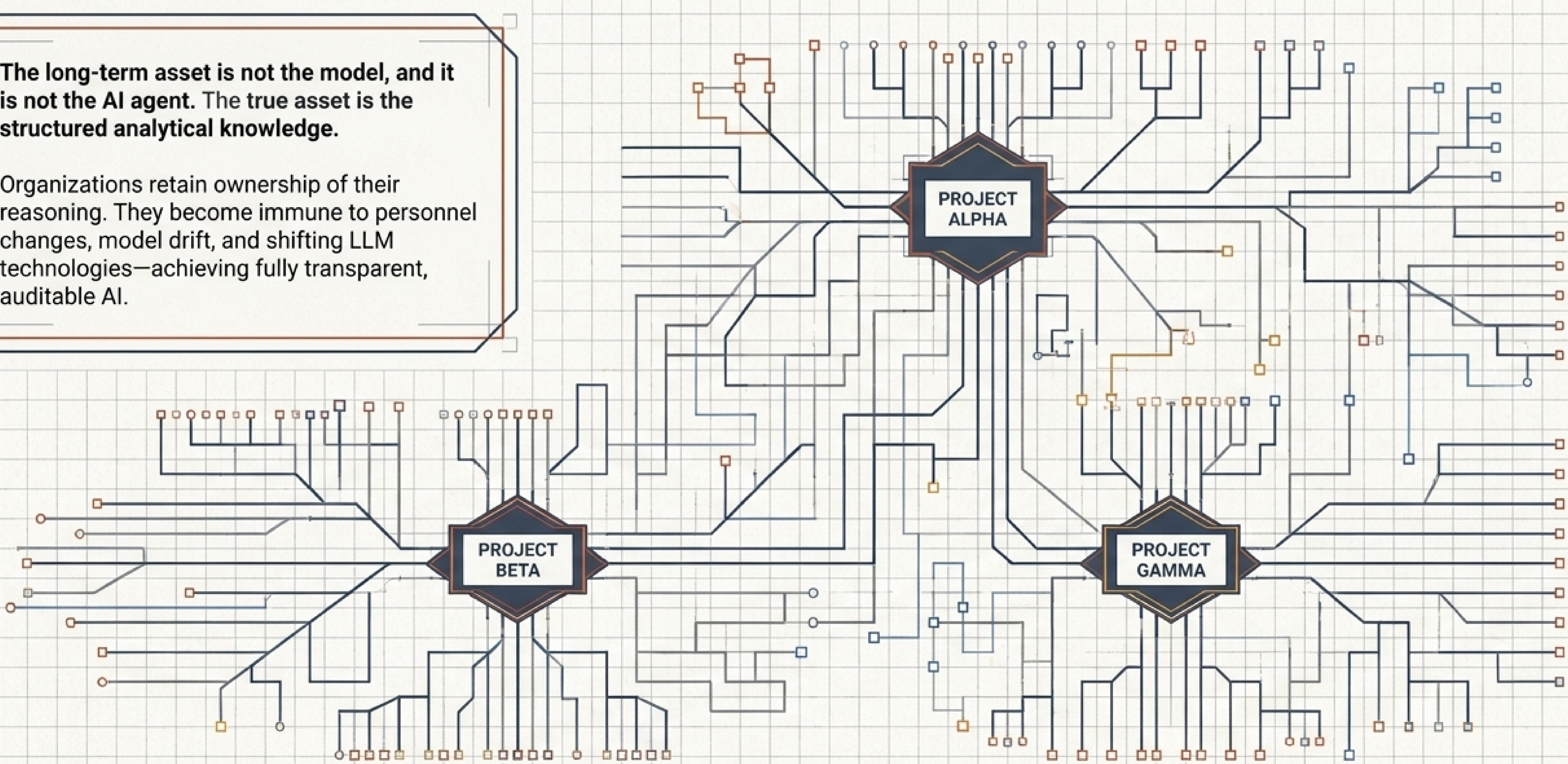
1. Target Setup: Human selected **borrower coordinates**.
2. Feature Engine: Generated **cluster-distance metrics**.
3. Modeling Engine: Trained **gradient boosting candidates**.
4. Agent Assembly: Serialized an **auditable artifact**.

NotebookLM

Preserving Institutional Memory

The long-term asset is not the model, and it is not the AI agent. The true asset is the **structured analytical knowledge.**

Organizations retain ownership of their reasoning. They become immune to personnel changes, model drift, and shifting LLM technologies—achieving fully transparent, auditable AI.



NotebookLM